@FunctionalInterface
public interface Runnable {
public abstract void run();
}

class Go {
static void go() { System.out.println("Go:go()"); }
}

public Thread(Runnable target) {
init(null, target, "Thread-" + nextThreadNum(), 0);
}

public class Demo {
static INNERCLASS Demo$1 null null /⽅法1的匿名内部类
public final static INNERCLASS MethodHandles$Lookup MethodHandles Lookup /⽅法2创建lambda表达式⽤的内部类
public static main([LString;)V

/⽅法1：匿名内部类
L0
LINENUMBER 10 L0
NEW Thread /创建Thread对象thread1
DUP

NEW Demo$1 /创建匿名内部类对象d
DUP

INVOKESPECIAL Demo$1.<init> ()V /初始化d
INVOKESPECIAL Thread.<init> (LRunnable;)V /初始化thread1
L1
LINENUMBER 15 L1

INVOKEVIRTUAL Thread.start ()V /启动线程，thread1.run() > d.run()

/⽅法2：lambda 编译器没有将Lambda表达式转为匿名内部类（节省类加载开销，提升JVM运⾏性能）
L2
LINENUMBER 17 L2
NEW Thread /创建Thread对象thread2
DUP

/动态⽣成lambda内部类，实现Runnable接⼝ > 加载类 > 创建实例r
INVOKEDYNAMIC run()LRunnable; [
LambdaMetafactory.metafactory(LMethodHandles$Lookup;LString;LMethodType;LMethodType)LCallSite;
()V,
Demo.lambda$main$0()V,
()V
]
INVOKESPECIAL Thread.<init> (LRunnable;)V /初始化thread2
INVOKEVIRTUAL Thread.start ()V /启动线程，thread2.run() > r.run()

/⽅法3：⽅法引⽤
L3
LINENUMBER 19 L3
NEW Thread
DUP

INVOKEDYNAMIC run()LRunnable; [//参数⾥指定需要实现的接⼝，⽅法，和实际执⾏的⽅法
LambdaMetafactory.metafactory(…)LCallSite;
()V,
Go.go()V,
()V
]
INVOKESPECIAL Thread.<init> (LRunnable;)V
INVOKEVIRTUAL Thread.start ()V
L4
LINENUMBER 18 L4
RETURN
L5

LOCALVARIABLE args [LString; L0 L4 0
MAXSTACK = 4
MAXLOCALS = 1

/lambda表达式被编译成了Demo的私有静态⽅法
private static synthetic lambda$main$0()V
L0
LINENUMBER 17 L0
GETSTATIC System.out : LPrintStream;
LDC "using lambda expression"
INVOKEVIRTUAL PrintStream.println (LString;)V
RETURN
MAXSTACK = 2
MAXLOCALS = 0
}

src/hotspot/share/interpreter/zero/bytecodeInterpreter.cpp:run() //解释器执⾏bytecode
> switch(opcode) /读取opcode，执⾏相关操作
CASE(_invokedynamic): {
u4 index = Bytes:get_native_u4(pc+1);
ConstantPoolCacheEntry* cache = cp >constant_pool() >invokedynamic_cp_cache_entry_at(index);
/1.解析符号引⽤，找到Method
if (! cache >is_resolved((Bytecodes:Code) opcode)) {
CALL_VM(InterpreterRuntime:resolve_invokedynamic(THREAD),handle_exception);
cache = cp >constant_pool() >invokedynamic_cp_cache_entry_at(index);
}
/2.lambda内部类对象r⼊栈
if (cache >has_appendix()) {
ConstantPool* constants = METHOD >constants();
SET_STACK_OBJECT(cache >appendix_if_resolved(constants), 0);
MORE_STACK(1);
}
/3.设置调⽤信息
Method* method = cache >f1_as_method();
istate >set_msg(call_method);
istate >set_callee(method);
istate >set_callee_entry_point(method >from_interpreted_entry());
istate >set_bcp_advance(5);
BI_PROFILE_UPDATE_CALL();
UPDATE_PC_AND_RETURN(0);
}

public class Demo {
public static void main(String[] args) {
/⽅法1：匿名内部类实现Runnable
new Thread(new Runnable() {
@Override
public void run() {
System.out.println("using anonymous class”);
}
}).start();

/⽅法2：lambda表达式assign给Runnable接⼝，jvm完成lambda > Runnable对象的封装
new Thread(() > System.out.println("using lambda expression”)).start();

/⽅法3：Method Reference assign给Runnable接⼝，jvm完成funtion > Runnable对象的封装
new Thread(Go:go).start(); /assign functions to interfaces
* If you assign a method reference or a lambda expression to the target interface,
java will adapt your assignment to the target interface by wrapping your method or
lambda expression in an instance of a class that implements the target interface. /
}
}
/运⾏结果
using lambda expression
using anonymous class
Go:go()

LinkResolver:resolve_invoke() > resolve_invokedynamic() > resolve_dynamic_call()
> methodHandle SystemDictionary:find_dynamic_call_site_invoker() {
.

JavaValue result(T_OBJECT);
/1.动态⽣成lambda内部类，实现functional interface > 加载内部类 > 创建内部类实例inst
JavaCalls:call_static(&result, SystemDictionary:MethodHandleNatives_klass(),
vmSymbols:linkCallSite_name(), vmSymbols:linkCallSite_signature(), &args);
Handle mname(THREAD, (oop) result.get_jobject()); /inst
/返回inst的run⽅法执⾏⼊⼝
return unpack_method_and_appendix(mname, caller, appendix_box, appendix_result, THREAD);
}

MemberName linkCallSiteImpl(Class<?> caller, MethodHandle bootstrapMethod, String name,
MethodType type, Object staticArguments, Object[] appendixResult) {
CallSite callSite = CallSite.makeSite(bootstrapMethod,name,type,staticArguments,caller);
appendixResult[0] = callSite.dynamicInvoker(); /Demo$$Lambda$1实例
return Invokers.linkToTargetMethod(type);
}

CallSite LambdaMetafactory.metafactory() > CallSite InnerClassLambdaMetafactory.buildCallSite() {
/1.generate a class file which implements the functional interface > define the class
final Class<?> innerClass = spinInnerClass();
Constructor<?>[] ctrs = innerClass.getDeclaredConstructors();
Object inst = ctrs[0].newInstance(); //2.创建Demo$$Lambda$1实例
return new ConstantCallSite(MethodHandles.constant(samBase, inst)); /3.创建callsite
}

import java.lang.invoke.LambdaForm.Hidden;
/ $FF: synthetic class
final class Demo$$Lambda$1 implements Runnable { //1.实现Runnable接⼝
private Demo$$Lambda$1() {} //2.空构造函数
@Hidden
public void run() { //3.实现Runanble.run()
Demo.lambda$main$0(); //4.调⽤lambda表达式的⽅法体
}
}

MemberName MethodHandleNatives.linkCallSite(Object callerObj,Object bootstrapMethodObj, Object nameObj,

Object typeObj,Object staticArguments,Object[] appendixResult) {
MethodHandle bootstrapMethod = (MethodHandle)bootstrapMethodObj; /LambdaMetafactory.metafactory()
Class<?> caller = (Class<?>)callerObj; /Demo的mirror
String name = nameObj.toString().intern(); /run
MethodType type = (MethodType)typeObj; /()Runnable
return linkCallSiteImpl(caller, bootstrapMethod, name, type,staticArguments, appendixResult);
}

/ First time execution: Resolve symbols, create a permanent CallSite object.
InterpreterRuntime:resolve_invokedynamic(JavaThread* thread) {
CallInfo info;
constantPoolHandle pool(thread, method(thread) >constants());
int index = get_index_u4(thread, bytecode);
LinkResolver:resolve_invoke(info, Handle(), pool,index, bytecode, CHECK); /1.符号引⽤>直接引⽤
ConstantPoolCacheEntry* cp_cache_entry = pool >invokedynamic_cp_cache_entry_at(index);
cp_cache_entry >set_dynamic_call(pool, info); /2.直接引⽤缓存到常量池
}

Lambda表达式实现原理（⾸次调⽤时动态链接⽅法调⽤，⽆需在编译器⽣成.class⽂件，减少类加载器的负担）
1.Demo.java：main()中调⽤f()，⼊参是@FunctionalInterface，传⼊具体动作Lambda表达式：(parameters) > body
2.Demo.class：Lambda表达式被编译成Demo的私有静态⽅法lambda$main$0()
3.Demo.class：调⽤f()的操作编译成2个字节码命令
> INVOKEDYNAMIC:将lambda$main$0()包装到对象中（这些细节都是对programmer不可⻅的）
> INVOKESPECIAL:执⾏f()
4.执⾏INVOKEDYNAMIC:
> 动态⽣成lambda内部类：实现Runnable接⼝，run()中执⾏Lambda表达式的内容Demo.lambda$main$0()
> 加载类 > 创建实例r > 放⼊操作数栈顶
5.执⾏INVOKESPECIAL:
> 执⾏f()：获取栈顶r > r.run() > Demo.lambda$main$0()

interface Callable {
void call(String s);
}

class Describe {
void show(String msg) { System.out.println(msg); }
}

public class Demo {
public static void main(String[] args) {
Describe d = new Describe();
/合成lambda内部类A，实现Callable，d为成员变量，call()中调⽤d.show()
/创建A实例，保存到c
Callable c = d:show;
c.call("*******");
}
}

public class MethodReferences {
public final static INNERCLASS MethodHandles$Lookup MethodHandles Lookup
public static main([LString;)V
L0
LINENUMBER 13 L0
NEW Describe
DUP

INVOKESPECIAL Describe.<init> ()V
ASTORE 1
L1
LINENUMBER 14 L1
ALOAD 1 /d
DUP

INVOKEVIRTUAL Object.getClass ()LClass; /d.getClass()
POP

INVOKEDYNAMIC call(LDescribe;)LCallable; [
LambdaMetafactory.metafactory(LMethodHandles$Lookup;.)LCallSite;
(LString;)V,
Describe.show(LString;)V,
(LString;)V]
ASTORE 2 /c
L2
LINENUMBER 15 L2
ALOAD 2
LDC "*******"

INVOKEINTERFACE Callable.call (LString;)V (itf)
L3
LINENUMBER 16 L3
RETURN
L4

LOCALVARIABLE args [LString; L0 L4 0
LOCALVARIABLE d LDescribe; L1 L4 1
LOCALVARIABLE c LCallable; L2 L4 2
MAXSTACK = 2
MAXLOCALS = 3
}

final class MethodReferences$$Lambda$1 implements Callable {
private final Describe arg$1; /d
private MethodReferences$$Lambda$1(Describe var1) { this.arg$1 = var1; }
private static Callable get$Lambda(Describe var0) { return new MethodReferences$$Lambda$1(var0); }
@Hidden
public void call(String var1) {
this.arg$1.show(var1); /d.show(var1)
}
}

final class Demo$$Lambda$2 implements Runnable { //实现⽬标接⼝
private Demo$$Lambda$2() {}
@Hidden
public void run() {
Go.go(); //⽅法实现⾥调⽤指定⽅法
}
}

/1.lambda表达式引⽤外部类field：
class Closure1 {
int i;
IntSupplier makeFun(int x) {
return () > x + i +;
}
}

public class SharedStorage {
public static void main(String[] args) {
Closure1 c1 = new Closure1();
IntSupplier f1 = c1.makeFun(0);
IntSupplier f2 = c1.makeFun(0);
System.out.println(f1.getAsInt());
System.out.println(f2.getAsInt());
}
}
/运⾏结果
0
1

final class Closure1$$Lambda$1 implements IntSupplier {
private final Closure1 arg$1; /1.lambda内部类持有外部类实例的引⽤
private final int arg$2;
private Closure1$$Lambda$1(Closure1 var1, int var2) {
this.arg$1 = var1; /保存外部类实例c1的引⽤
this.arg$2 = var2; /x = 传⼊的int值
}
@Hidden
public int getAsInt() {
/执⾏lambda表达式内容，实时GET c1.i的值
return this.arg$1.lambda$makeFun$0(this.arg$2);
}
}

private synthetic lambda$makeFun$0(I)I
L0
LINENUMBER 8 L0
ILOAD 1
ALOAD 0
DUP

GETFIELD functionalProgramming/Closure1.i : I /1.读取c1.i
/2.执⾏业务逻辑
DUP_X1
ICONST_1
IADD

PUTFIELD functionalProgramming/Closure1.i : I
IADD
IRETURN
L1

LOCALVARIABLE this LfunctionalProgramming/Closure1; L0 L1 0
LOCALVARIABLE x I L0 L1 1
MAXSTACK = 5
MAXLOCALS = 2

/2.lambda表达式引⽤当前method effectively final local varaible
class Closure2 {
IntSupplier makeFun(int x) {
int i;
return () > x + i; /x和i是effectively final的
}
}

public class SharedStorage {
public static void main(String[] args) {
Closure1 c1 = new Closure1();
IntSupplier f1 = c1.makeFun(0);
IntSupplier f2 = c1.makeFun(0);
System.out.println(f1.getAsInt());
System.out.println(f2.getAsInt());
}
}
/运⾏结果
0
0

final class Closure2$$Lambda$1 implements IntSupplier {
private final int arg$1; /1.lambda内部类保存local variable的值
private final int arg$2;
private Closure1$$Lambda$1(int var1, int var2) {
this.arg$1 = var1; /保存local varialbe的值
this.arg$2 = var2; /保存local varialbe的值
}
@Hidden
public int getAsInt() {
/执⾏lambda表达式内容，⼊参是local variables的值
return Closure2.lambda$makeFun$0(this.arg$1, this.arg$2);
}
}

private static synthetic lambda$makeFun$0(II)I
L0
LINENUMBER 8 L0
ILOAD 0 /⼊参x的值
ILOAD 1 /⼊参i的值
IADD
IRETURN
L1
LOCALVARIABLE x I L0 L1 0
LOCALVARIABLE i I L0 L1 1
MAXSTACK = 2
MAXLOCALS = 2

/3.lambda表达式引⽤当前method final local varaible
class Closure3 {
IntSupplier makeFun(final int x) {
final int i = 9;
return () > x + i;
}
}

public class SharedStorage {
public static void main(String[] args) {
Closure1 c1 = new Closure1();
IntSupplier f1 = c1.makeFun(0);
IntSupplier f2 = c1.makeFun(0);
System.out.println(f1.getAsInt());
System.out.println(f2.getAsInt());
}
}
/运⾏结果
9
9

final class Closure3$$Lambda$1 implements IntSupplier {
private final int arg$1; /1.lambda内部类保存local variable的值
private Closure1$$Lambda$1(int var1) {
this.arg$1 = var1; /保存local varialbe的值
}
@Hidden
public int getAsInt() {
/执⾏lambda表达式内容，⼊参是local variables的值
return Closure3.lambda$makeFun$0(this.arg$1);
}
}

private static synthetic lambda$makeFun$0(I)I
L0
LINENUMBER 8 L0
ILOAD 0
BIPUSH 9 /i的值在编译lambda⽅法时写死了
IADD
IRETURN
L1
LOCALVARIABLE x I L0 L1 0
MAXSTACK = 2
MAXLOCALS = 1

public class AnonymousClosure {
IntSupplier makeFun(int x) {
int i = 0;
return new IntSupplier() { /anonymous inner class as closure
public int getAsInt() { return x + i; }
};
}
}